

RETAIL SALES & INVENTORY ANALYTICS PLATFORM

Microsoft Fabric • OneLake • PySpark • Power BI

Beginner-Friendly Project Report

Screenshots are matched to the exact implementation step they demonstrate.

1. Project Overview

This project builds an end-to-end retail analytics solution using Microsoft Fabric. Orders, Returns and Inventory data are stored, processed with PySpark, organized into Bronze/Silver/Gold layers, and presented through a Power BI dashboard.

Project objectives

- Store retail source data in a structured Fabric environment.
- Create a Bronze layer for raw data.
- Clean and standardize data in the Silver layer.
- Create product-level Gold KPIs.
- Connect the Gold dataset to Power BI.
- Build an interactive retail performance dashboard.

Architecture

Source	Bronze	Silver	Gold	Power BI
Orders Returns Inventory	Raw files	Cleaned data	Product KPIs	Dashboard

2. Step-by-Step Implementation

Step 1 — Prepare the source storage

The first screenshot shows the Azure/Fabric storage area containing the source files. This is the starting point before the data is processed.

1. Open the storage area used for the retail project.
2. Verify that the required source files are present.
3. Keep the source data available before beginning the pipeline.

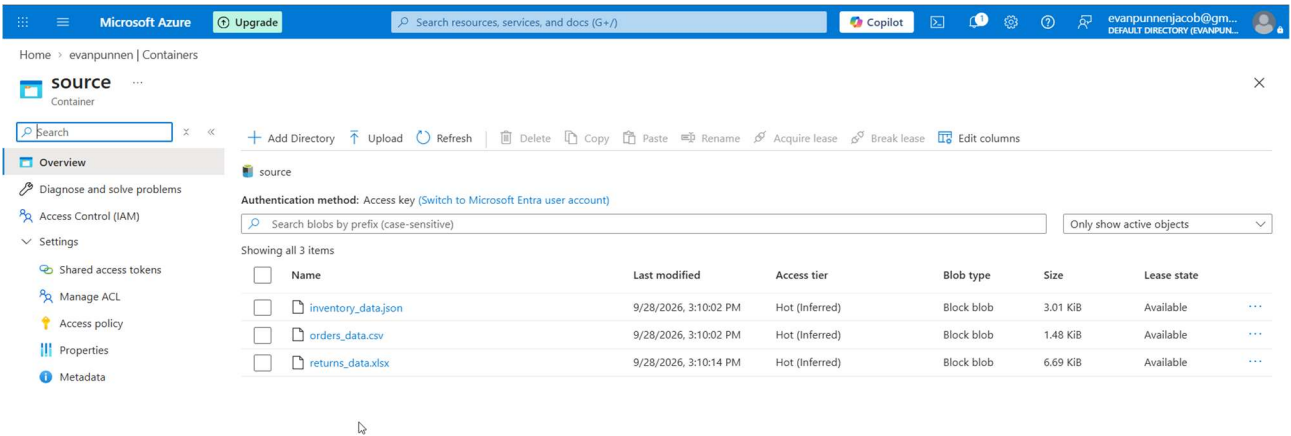


Figure 1 — Source storage area showing the project input files.

Step 2 — Run the data pipeline

The second screenshot shows the Fabric pipeline after execution. The pipeline contains the project data-flow stages and shows a successful run.

4. Open the Fabric pipeline.
5. Check that the required activities are connected.
6. Run the pipeline.

- Open the output/activity details and confirm the activities completed successfully.

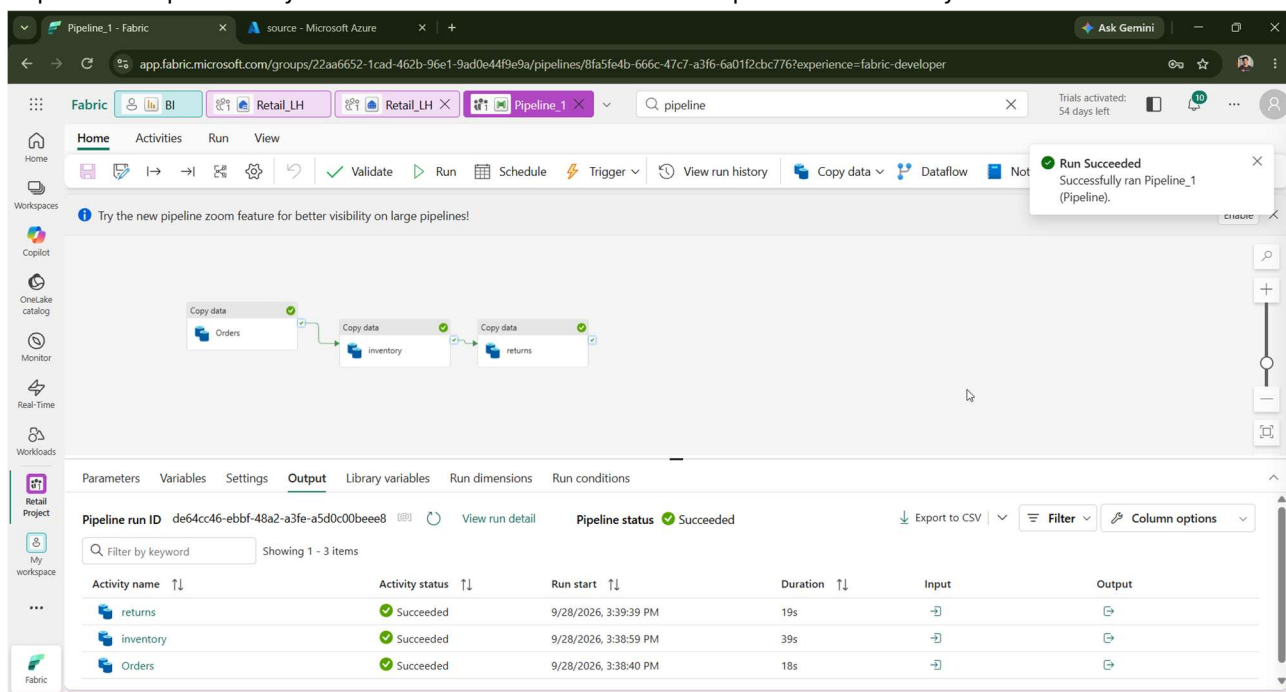


Figure 2 — Fabric pipeline with a successful run and completed activities.

Step 3 — Load and inspect Inventory data in the Notebook

The third screenshot shows the Fabric Notebook loading Inventory data with PySpark and displaying the resulting table.

- Open the Fabric Notebook.
- Load the Inventory source into a PySpark DataFrame.
- Display the DataFrame to inspect Product ID, Product Name, stock, dates, warehouse, cost price and availability.
- Use this inspection to confirm the actual source schema before cleaning.

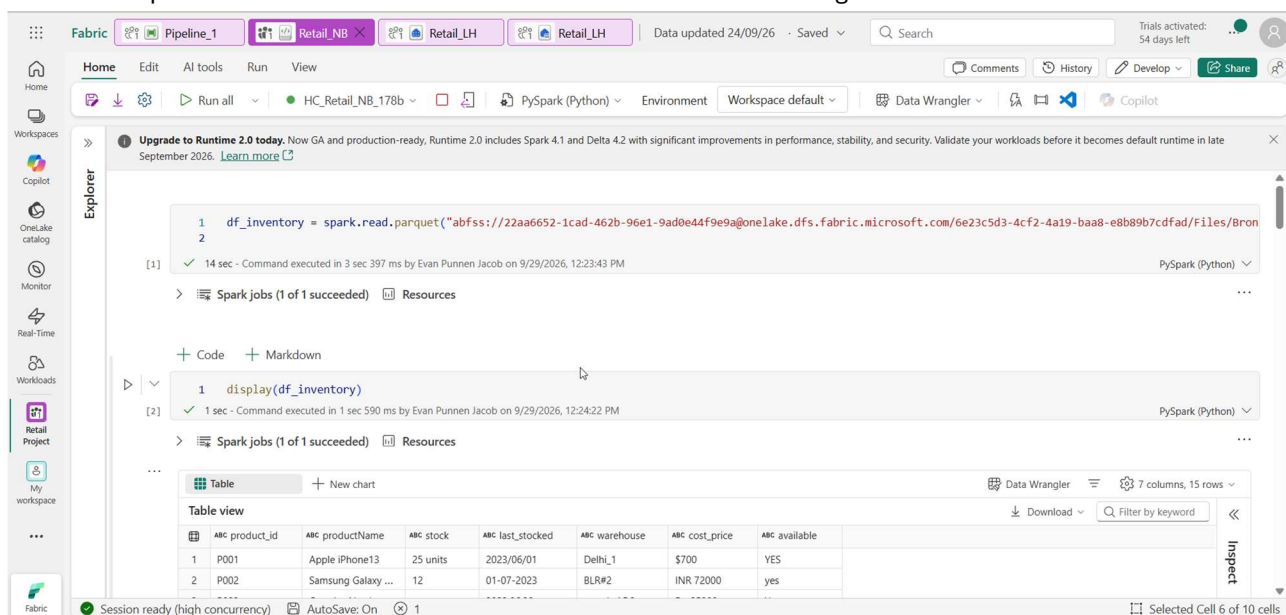


Figure 3 — Fabric Notebook loading and displaying the Inventory DataFrame.

Step 4 — Verify the Bronze layer

The fourth screenshot shows the Bronze folder inside the Lakehouse. The raw Orders, Returns and Inventory files are visible here.

- Open the Lakehouse Explorer.
- Open Files → Bronze.

14. Verify that Orders, Returns and Inventory raw files are present.

15. Do not overwrite the raw files with cleaned data.

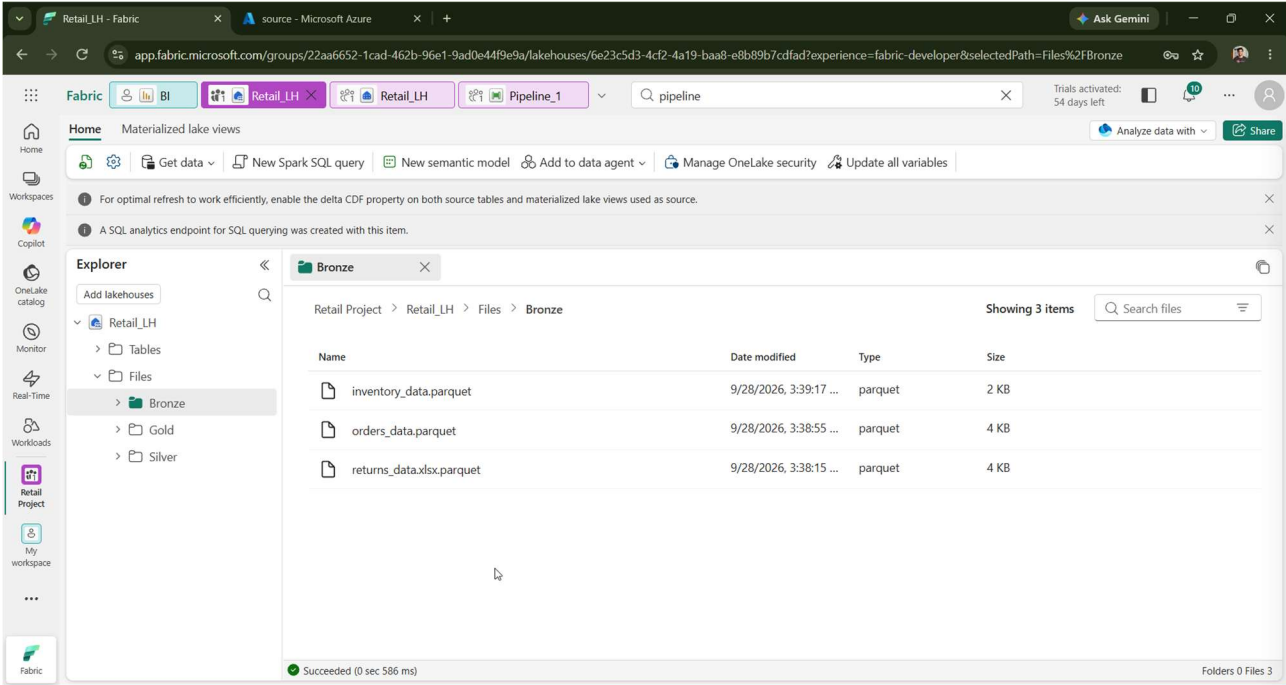


Figure 4 — Bronze folder containing the raw Orders, Returns and Inventory files.

Step 5 — Inspect the Bronze Inventory data

The fifth screenshot shows the actual Inventory file opened from the Bronze area. This step is used to inspect the raw values before Silver transformation.

16. Open the Inventory Bronze file.

17. Check the actual column names and values.

18. Identify fields such as product_id, productName, stock, last_stocked, warehouse, cost_price and available.

19. Use the observed structure when writing the Silver cleaning logic.

A screenshot of the Microsoft Fabric web interface showing the 'inventory_data.parquet (preview)' table. The table is displayed with 15 rows and 8 columns. The columns are: product_id, productName, stock, last_stocked, warehouse, cost_price, and available. The data includes various product names like 'Apple iPhone13', 'Samsung Galaxy S21', 'Oneplus Nord', 'Nokia 3310', 'Xiaomi note 12 pro', 'Vivo V25', 'Motorola Edge-40', 'LG Velvet', 'Realme X50', 'Asus ROG Phone', 'Google Pixel 6a', 'Sony Xperia 5', 'Nothing Phone 2', 'IQOO Neo 7', and 'Infinix Zero 5G'. The 'stock' column contains values like '25 units', '12', 'twenty', 'fifteen', 'thirty', 'twenty five', '35', '22', 'null', 'eighteen', and '10'. The 'last_stocked' column contains dates like '2023/06/01', '01-07-2023', '2023.06.30', '2023/07/05', '07-12-2023', '2023-07-20', '25.07.2023', '2023-06-18', '06-07-2023', '2023.07.10', '07/15/2023', '15-07-2023', '2023/07/18', '2023-07-22', and '23/07/2023'. The 'warehouse' column contains values like 'Delhi_1', 'BLR#2', 'mumbai@3', 'NULL', 'hyd/zone2', 'chn', 'Delhi_1', 'pune#city', 'gurgaon-56', 'bangalore!', 'Hyderabad\$', 'coimbatore@', 'delhi\$', 'chd!', and 'kolkata#'. The 'cost_price' column contains values like '\$700', 'INR 72000', 'Rs. 25000', 'NULL', '\$299', '350USD', 'USD500', '₹32000', 'USD 399', 'Rs.59999', '\$449', 'Rs 65000', '\$599', 'Rs.32000', and '\$250'. The 'available' column contains values like 'YES', 'yes', 'Y', 'NULL', 'True', 'NO', 'false', 'Yes', 'YES', 'Y', 'yes', 'Yes', 'True', 'Yes', and 'yes'.

	ABC product_id	ABC productName	ABC stock	ABC last_stocked	ABC warehouse	ABC cost_price	ABC available
1	P001	Apple iPhone13	25 units	2023/06/01	Delhi_1	\$700	YES
2	P002	Samsung Galaxy S21	12	01-07-2023	BLR#2	INR 72000	yes
3	P003	Oneplus Nord	twenty	2023.06.30	mumbai@3	Rs. 25000	Y
4	P004	Nokia 3310		2023/07/05		NULL	
5	P005	Xiaomi note 12 pro	55	07-12-2023	hyd/zone2	\$299	True
6	P006	Vivo V25	NULL	2023-07-20	chn	350USD	NO
7	P007	Motorola Edge-40	0	25.07.2023	Delhi_1	USD500	false
8	P008	LG Velvet	fifteen	2023-06-18	pune#city	₹32000	Yes
9	P009	Realme X50	30	06-07-2023	gurgaon-56	USD 399	YES
10	P010	Asus ROG Phone	twenty five	2023.07.10	bangalore!	Rs.59999	Y
11	P011	Google Pixel 6a	35	07/15/2023	Hyderabad\$	\$449	yes
12	P012	Sony Xperia 5	22	15-07-2023	coimbatore@	Rs 65000	Yes
13	P013	Nothing Phone 2	null	2023/07/18	delhi\$	\$599	True
14	P014	IQOO Neo 7	eighteen	2023-07-22	chd!	Rs.32000	Yes
15	P015	Infinix Zero 5G	10	23/07/2023	kolkata#	\$250	yes

Figure 5 — Raw Inventory data inspected inside the Bronze layer.

Step 6 — Create and validate the Gold KPI table

The sixth screenshot shows the Gold product KPI table. This is the business-ready output produced after the Silver data has been transformed and aggregated.

- 20. Load the cleaned Silver Orders, Returns and Inventory data.
- 21. Aggregate Returns by Order ID before joining to Orders.
- 22. Join Inventory information using the cleaned ProductName.
- 23. Calculate COGS as Quantity × Cost Price.
- 24. Calculate Net Profit as Order Amount – COGS.
- 25. Create Profit_Status to identify profit, loss and missing-data conditions.
- 26. Aggregate the data by ProductName to create product-level KPIs.
- 27. Save the result as gold_product_kpis.

Avg_Order_ID	Total_COGS	Avg_Cost	Net_Profit	Total_Returns	Returned_Order_ID	Total_Returns	Current_Stock	Profit_Status	Return_Rate
1	NULL	NULL	NULL	1	1	1099	NULL	Cost Data Missing	100
700	700	700	99	1	1	799	25	Profit	100
179997	59999	59999	-119998	1	1	59999	25	Loss	100
449	449	449	0	1	1	449	35	Profit	100
9	NULL	NULL	NULL	1	1	1099	NULL	Cost Data Missing	100
64000	32000	32000	-29000	1	1	35000	15	Loss	100
NULL	500	500	NULL	1	1	550	0	Quantity Missing	100
25000	25000	25000	-24501	1	1	499	20	Loss	100
NULL	399	399	NULL	1	1	399	30	Quantity Missing	100
0	NULL	72000	NULL	1	1	1200	12	Quantity Missing	100
130000	65000	65000	-65000	1	1	65000	22	Loss	100
1050	350	350	-675	1	1	375	NULL	Loss	100

Figure 6 — Gold `gold\_product\_kpis` table showing product-level KPI fields and Profit_Status.

Step 7 — Verify the Lakehouse tables and folders

The seventh screenshot shows the Lakehouse Explorer with the project organized into Bronze, Silver and Gold areas and the corresponding tables.

- 28. Open the Lakehouse Explorer.
- 29. Expand the Bronze, Silver and Gold folders.
- 30. Verify that the transformed data is present.
- 31. Verify that the Silver tables and Gold KPI output are available.

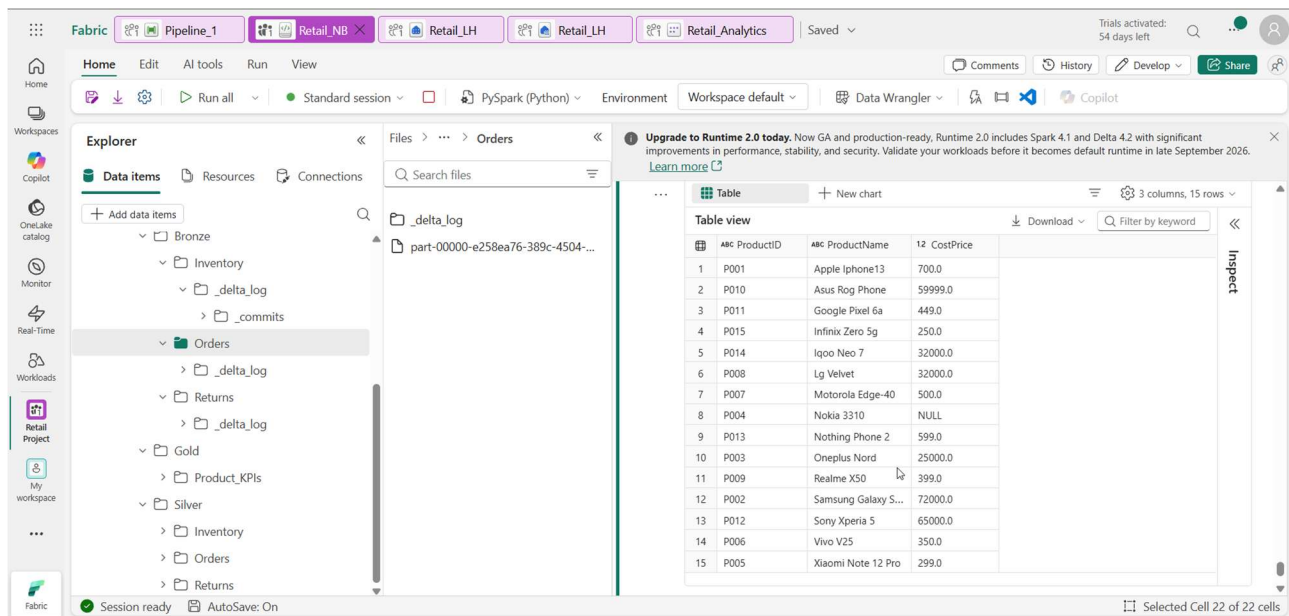


Figure 7 — Lakehouse Explorer showing the Bronze/Silver/Gold organization and project tables.

Step 8 — Connect the Gold table to Power BI

The eighth screenshot is specifically the Power BI model view. It shows the `gold\_product\_kpis` table and the fields available for report building.

32. Open the Power BI report in the Fabric workspace.
33. Open Model view.
34. Confirm that `gold\_product\_kpis` is available.
35. Check that fields such as Avg_Cost, Avg_Order_Value, Current_Stock, Net_Profit, ProductName, Return_Rate_Percent, Returned_Orders, Total_COGS and Total_Orders are visible.
36. Refresh the model after any Gold schema changes.

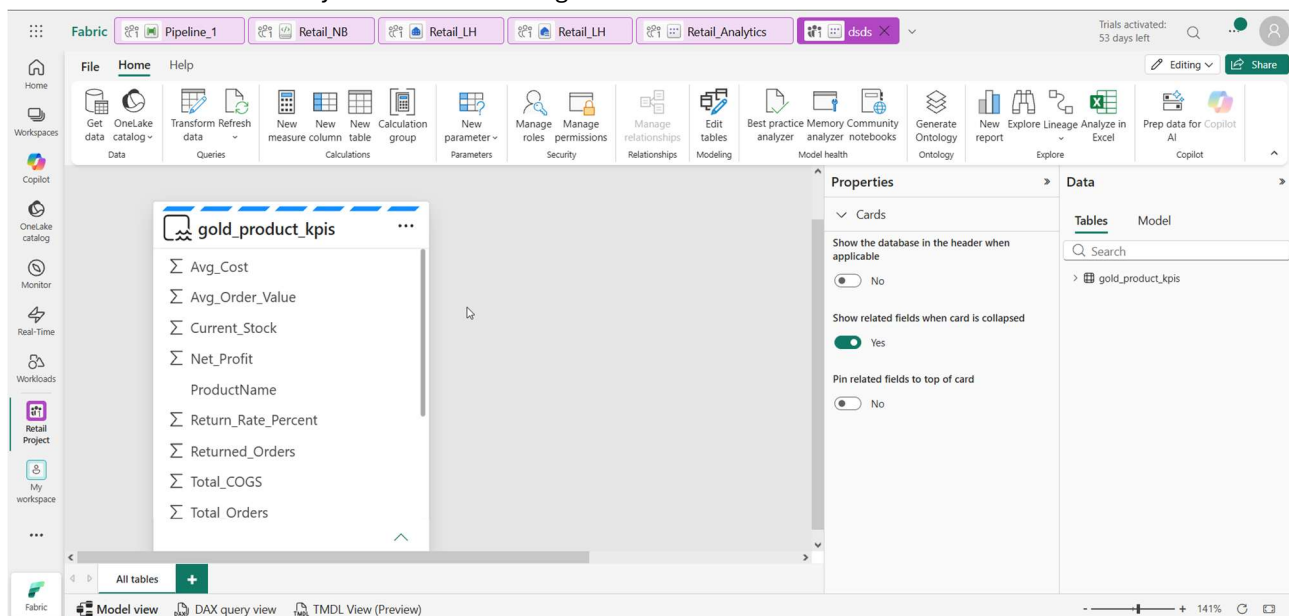


Figure 8 — Power BI Model view showing the `gold\_product\_kpis` table and its available fields.

Step 9 — Build the final Power BI dashboard

The ninth screenshot is the actual dashboard page. It shows the report visuals created from the Gold KPI table.

37. Add KPI cards for Total Revenue, Total Orders, Unique Customers, Net Profit and Return Rate %.
38. Add a ProductName filter/slicer.
39. Add a Profit_Status field/filter when required.

- 40. Add product-level visuals for revenue, profit, returns, stock and return amount.
- 41. Format the dashboard so the KPI cards and charts are easy to read.
- 42. Test the filters and confirm that the visuals respond to selections.

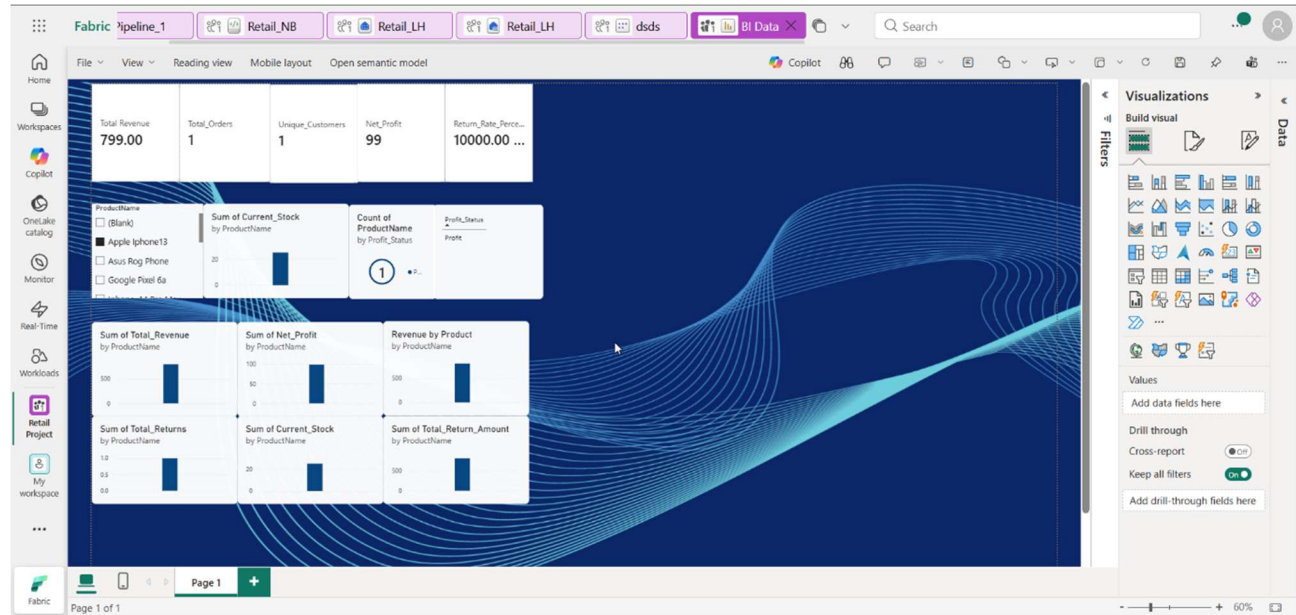


Figure 9 — Final Power BI Retail Analytics dashboard.

3. Gold KPI Calculations

The Gold layer is the reporting-ready layer. The main calculations used in the project are:

- $COGS = \text{Quantity} \times \text{Cost Price}$
- $\text{Net Profit} = \text{Order Amount} - COGS$
- $\text{Return Rate \%} = (\text{Returned Orders} \div \text{Total Orders}) \times 100$
- Profit_Status identifies Profit, Loss, Cost Data Missing or Quantity Missing.

Returns are summarized by Order ID before the Orders join. This prevents multiple return rows for one order from unnecessarily duplicating revenue.

4. Power BI Dashboard Components

- KPI Cards: Total Revenue, Total Orders, Unique Customers, Net Profit and Return Rate %.
- Filters: Product and Profit Status.
- Analysis: Revenue by Product, Net Profit by Product, Returns by Product, Return Amount by Product, Current Stock by Product and Quantity Sold by Product.
- Summary: Products by Profit Status.

5. Beginner Explanation

Layer	Meaning	Purpose
Bronze	Raw data	Keep the incoming data.
Silver	Clean data	Fix names, types, dates, nulls and duplicates.
Gold	Business data	Create KPIs and reporting-ready data.

6. Technologies Used

- Microsoft Fabric
- OneLake
- Fabric Lakehouse
- Fabric Notebook
- PySpark

- Power BI

7. Interview Explanation

I built a retail analytics platform using Microsoft Fabric. I organized Orders, Returns and Inventory data into a Bronze, Silver and Gold architecture. I used PySpark in a Fabric Notebook to inspect, clean and transform the data, joined the datasets, calculated COGS and Net Profit, and created a product-level Gold KPI table. Finally, I connected the Gold table to Power BI and built a dashboard for revenue, orders, customers, profitability, returns and inventory analysis.

8. Final Validation Checklist

- Source files are available.
- Pipeline execution is successful.
- Bronze files are present.
- Inventory source data has been inspected.
- Gold `gold\_product\_kpis` exists.
- Power BI Model view contains the Gold fields.
- Power BI dashboard is connected to the Gold dataset.
- Dashboard filters and visuals work as expected.